

VAHAK

Offline-First Public Bus Parcel Delivery Network — Technical Documentation

Smart Kopargaon Hackathon 2026 · Problem Statement PS014 · Round 2 Submission

1. Problem Statement

Parcel movement in rural and semi-urban areas is often slow, expensive, or unavailable outside major courier networks. Public buses already travel fixed routes on fixed schedules through these same areas, with spare load capacity going unused on most trips.

2. Proposed Solution

Vahak coordinates parcel booking, bus-route assignment, conductor acceptance, and recipient verification through a role-based consignment workflow layered on top of an existing public bus network. The system does not operate vehicles; it models routes, halts and buses as data and assigns parcels to that existing capacity.

3. Motivation

Three gaps motivate the design: (a) existing transport capacity goes unused for logistics, (b) an unattended handover between strangers (conductor → recipient) needs a trust mechanism, and (c) four distinct actors (sender, conductor, recipient, admin) need clearly separated permissions rather than one generic user type.

4. Objectives

- Model routes, halts and buses as first-class, queryable network entities.
- Let senders book against real route and halt data, not free-text addresses.
- Give conductors an explicit accept → handover flow tied to consignment state.
- Verify delivery through a single-use, time-limited PIN rather than an honor system.
- Give administrators direct visibility into and control over network infrastructure.

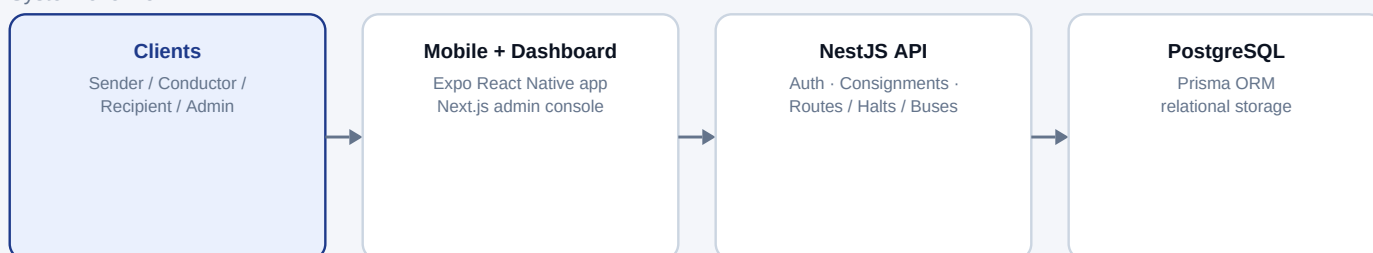
5. Key Features (implemented)

- Role-based authentication for four roles (JWT + bcrypt password hashing).
- Route / halt / bus network model with per-record data-source provenance (REAL, PUBLIC_THIRD_PARTY, COMMUNITY_DERIVED, DERIVED, SIMULATED) populated via an OSM ingestion pipeline.
- Explicit six-state consignment lifecycle with a server-written event log per transition.
- PIN-based delivery proof: SHA-256 hashed code, unique nonce, 30-minute expiry, attempt limiting.
- Admin web dashboard for route/halt/bus management and consignment lookup by tracking ID.

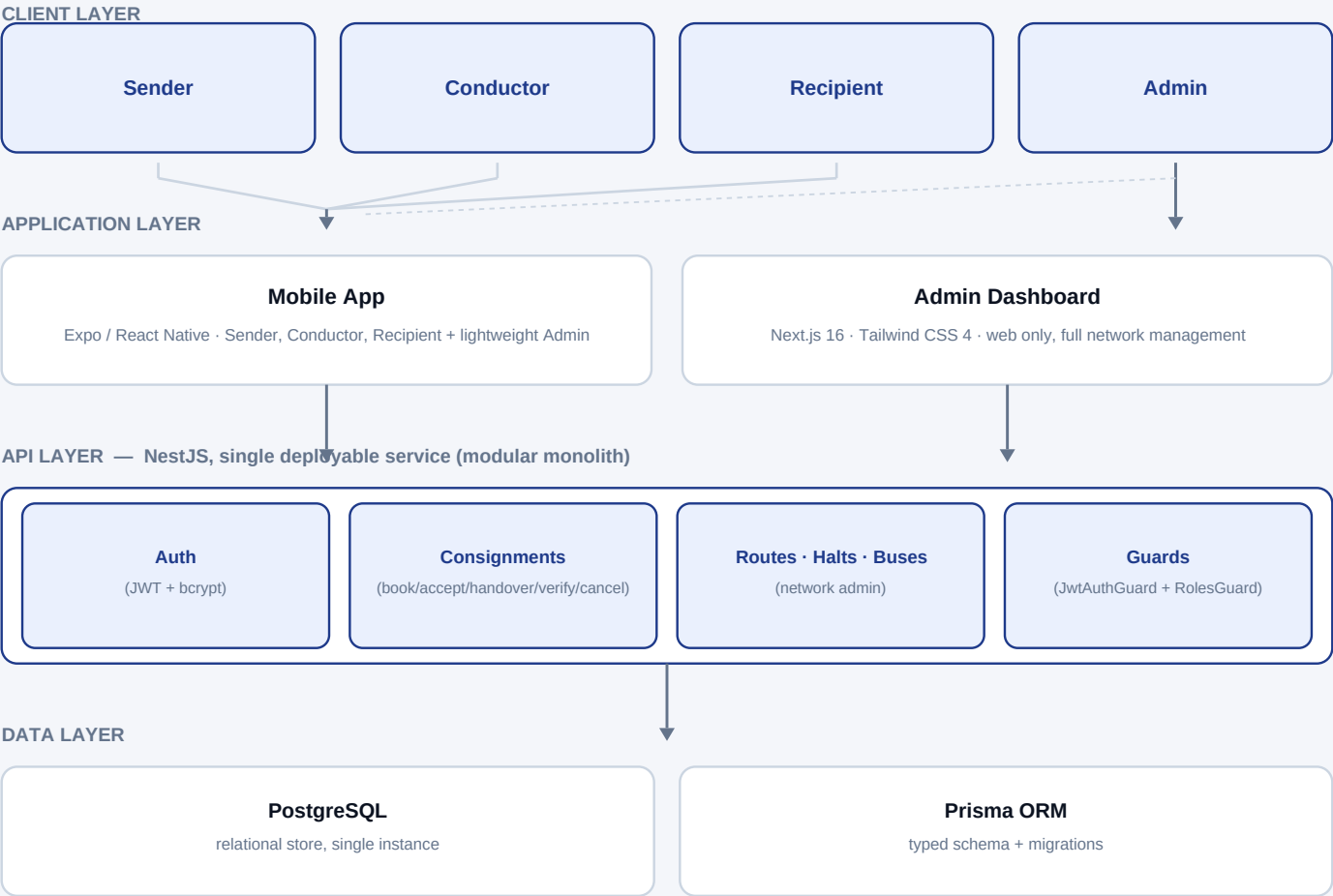
6. Technical Overview

Vahak is a single monorepo: apps/mobile (Expo / React Native, role-based navigation), apps/dashboard (Next.js admin console), services/api (NestJS REST API), and database/ (Prisma schema, migrations, seed and OSM-ingestion scripts). The current prototype is a connected, request/response system: the mobile client calls the API directly over REST for every read and write. Offline-first local persistence is a stated design direction for the product but is not implemented in this build — see Page 8.

System overview



System Architecture



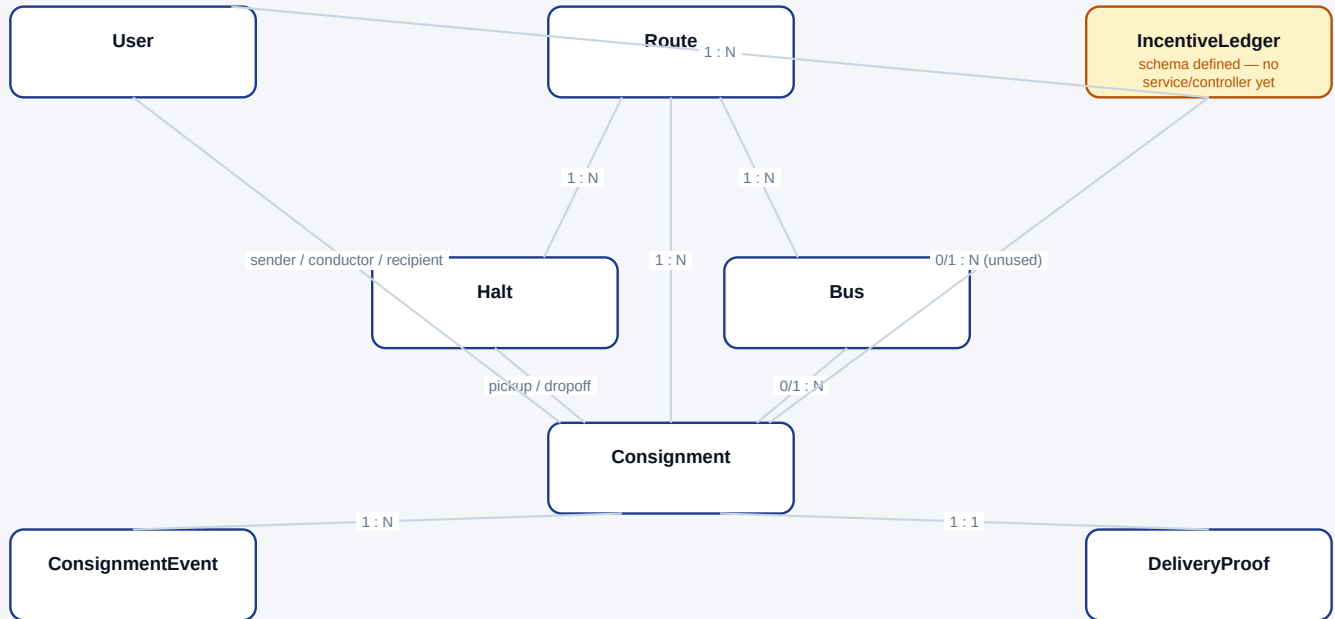
Layer	Technology	Purpose
Mobile app	Expo / React Native 0.86, React Navigation 7 (native-stack)	Role-based mobile client for Sender / Conductor / Recipient / Admin
Admin dashboard	Next.js 16, Tailwind CSS 4	Web console for network management and consignment lookup
Backend API	NestJS 11, TypeScript	Single modular REST service (auth, consignments, routes/halts/buses)
ORM	Prisma 7 (+ @prisma/adapters-pg)	Typed schema, migrations, query builder over PostgreSQL
Database	PostgreSQL	Relational storage; single instance
Auth	@nestjs/jwt + passport-jwt, bcrypt	Stateless bearer-token auth; salted password hashing

The backend is a single modular NestJS application — a modular monolith, not a microservice deployment. Feature modules (auth, consignments, routes/halts/buses) are separated by responsibility but share one process, one database connection pool, and one deploy artifact, which matches the current single-team, single-environment scope of the prototype. Role separation is enforced at the API boundary (JwtAuthGuard + RolesGuard with an explicit @Roles() decorator per endpoint), not by separate deployments per role.

Offline operation: not implemented in the current build. The mobile client is a standard connected REST client (axios); every screen requires network connectivity to the API. No local database, request queue, or sync layer exists in apps/mobile today.

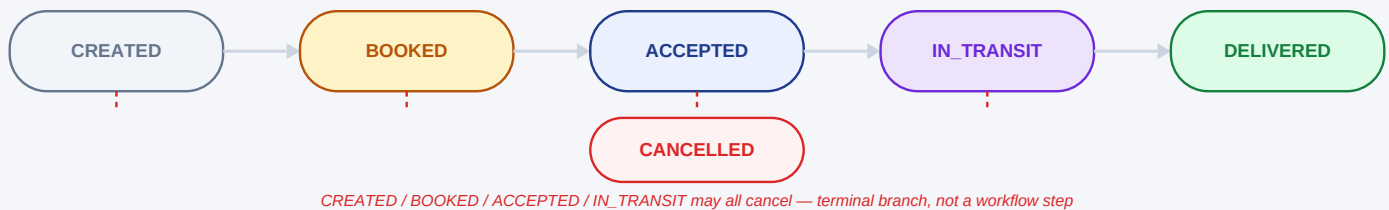
System Design & Data Model

Entity-relationship model (from database/prisma/schema.prisma)



Role	Implemented capability
Sender	Books a consignment against a route + pickup/dropoff halt pair; views and cancels own consignments (while in a cancellable status).
Conductor	Views consignments booked-but-unclaimed and those assigned to them; accepts a consignment; initiates handover (generates the PIN).
Recipient	Views incoming deliveries addressed to them; verifies handover by submitting the PIN.
Admin	Full CRUD on routes, halts and buses; read-only consignment lookup by tracking ID (no bulk listing endpoint exists).

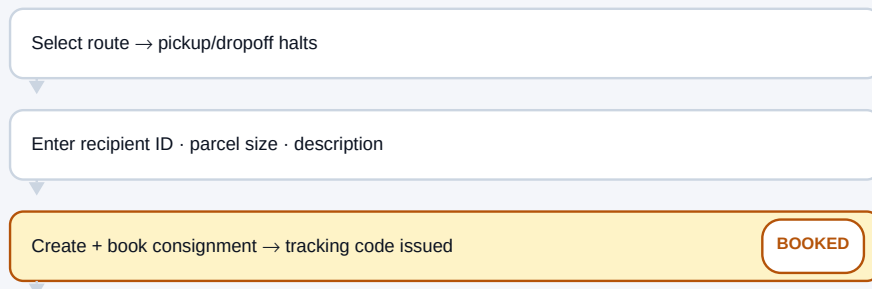
Consignment state machine



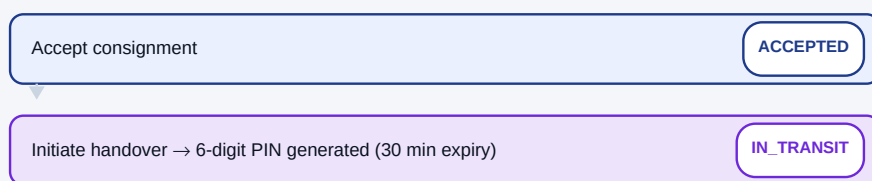
HANDOVER_INITIATED and HANDOVER_VERIFIED are recorded as ConsignmentEvent entries inside the IN_TRANSIT status — the Prisma schema defines only six values for ConsignmentStatus itself.

End-to-End Operational Workflow

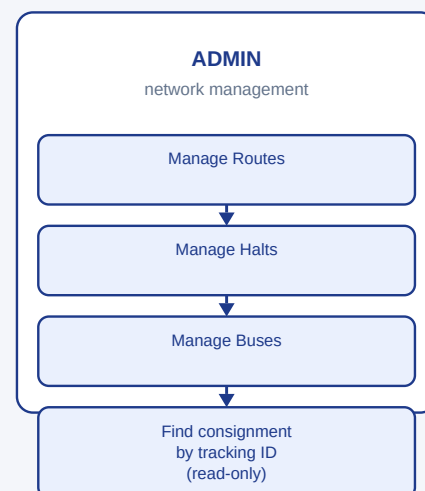
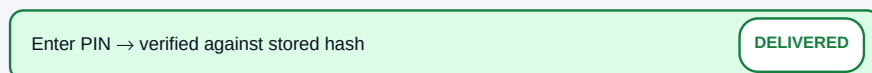
SENDER



CONDUCTOR



RECIPIENT



Methodology

- **Role-based access** — every mutating endpoint is guarded by JWT authentication plus a `@Roles ( )` check (e.g. only CONDUCTOR can call accept/handover).
- **Explicit state transitions** — each transition is its own endpoint with a server-side precondition (e.g. handover can only be initiated from ACCEPTED; verification only from IN_TRANSIT), rejected with a typed exception otherwise.
- **Route/halt-based logistics** — pickup and dropoff halts are validated server-side to belong to the selected route before a consignment is created.
- **Secure delivery verification** — the handover PIN is never stored in plain text: the API persists `SHA-256(pin:nonce)`, a unique nonce, a 30-minute expiry, and an attempt counter (capped at 5).
- **Offline-first synchronization** — not implemented in the current build; every step above requires live connectivity to the API.
- **Validation and error handling** — request bodies are validated with class-validator DTOs; domain failures map to typed HTTP exceptions (404 not found, 403 forbidden, 400 bad request, 409 conflict) through a shared exception filter.

IN_TRANSIT — semantic clarification

IN_TRANSIT is set when the conductor initiates handover (the PIN is generated), which happens near arrival at the dropoff halt — not when the bus departs with the parcel aboard. The schema has no location or telemetry field on Bus/Route; the status does not represent, and this document does not claim, live GPS movement or an ETA.

Implementation

Mobile application — implemented screens

Area	Screens
Auth	Login, Register (role selector: Sender / Conductor / Recipient)
Sender	Home, Available Routes, Select Halts, Book Parcel, Booking Confirmation, My Consignments, Consignment Detail
Conductor	Home, Available Consignments (sectioned queue), Find Consignment (by ID), Consignment Detail
Recipient	Home, Incoming Deliveries, Find Delivery (by ID), Consignment Detail, PIN Verification, Delivery Confirmation
Admin	Home, Route List/Form/Detail, Halt Form, Bus Form, Find Consignment (by ID), Consignment Detail (read-only)

Backend — modules

Module	Responsibility
Auth	Register, login, current-user lookup; bcrypt hashing; JWT issuance (1h expiry)
Consignments	create, book, accept, initiate-handover, verify-handover, cancel, per-role list/detail
Routes / Halts / Buses	Admin CRUD; sender-facing read-only route/halt discovery endpoints
Shared	PrismaService, global exception filter, request/response provenance stripping

Database

PostgreSQL accessed through Prisma 7. Six migrations track schema evolution from initial models through timestampz conversion, password support, delivery-proof attempt tracking, and route/halt/bus provenance metadata. Cascading deletes remove dependent `ConsignmentEvent` and `DeliveryProof` rows with their parent consignment, and dependent `Halt` rows with their parent route. Indexes exist on `Consignment` (`senderId`, `recipientId`, `conductorId`, `routeId`, `status`) and `ConsignmentEvent` (`consignmentId`, `createdAt`). Unique constraints cover tracking code, user phone/email, and `provider+externalId` pairs on ingested route/halt/bus records.

Security — implemented mechanisms only

- Passwords hashed with bcrypt (cost factor 12); raw passwords are never persisted.
- JWT bearer authentication (@nestjs/jwt), 1-hour expiry, payload limited to user id / phone / role.
- Per-endpoint role authorization via `JwtAuthGuard` + `RolesGuard`.
- Self-registration is restricted server-side to SENDER / CONDUCTOR / RECIPIENT (`RegisterDto` validator) — ADMIN accounts cannot be created through the public endpoint.
- Handover PIN stored only as `SHA-256(pin:nonce)`, with a unique nonce, 30-minute expiry, and a server-enforced 5-attempt cap; ownership is checked (only the assigned conductor can initiate handover, only the addressed recipient can verify it).

Not implemented: refresh tokens, request rate limiting, PIN regeneration/resend, and encryption beyond what the hosting/database layer provides by default (not independently verified in this review).

Testing & Validation

Automated tests present in repository

Jest specs exist for the modules below (static inspection of committed test files). Execution was not re-run as part of this documentation pass — this environment has no provisioned PostgreSQL instance or .env configured, so results are reported as “present in repository,” not as a fresh pass/fail run.

Test file	Module under test	Test cases (it-blocks)
auth.service.spec.ts / auth.controller.spec.ts	Registration, login	2
consignments.service.spec.ts	Full consignment lifecycle logic	39
routes.service.spec.ts, buses.service.spec.ts, halts.service.spec.ts	Route/halt/bus CRUD	6
sender-routes.service.spec.ts	Sender-facing route/halt discovery	7
provenance.spec.ts	Data-source provenance stripping	3
normalize.spec.ts, osm-write.spec.ts	OSM ingestion pipeline	34

Validation matrix

Method key: “Source inspection” = behavior confirmed by reading the guard/service/DTO logic in the repository, not by executing the app.

Feature	Verification method	Basis
Duplicate phone/email on registration is rejected	Source inspection	ConflictException in AuthService.register
Self-registration cannot create an ADMIN account	Source inspection	RegisterDto @IsIn(['SENDER', 'CONDUCTOR', 'RECIPIENT'])
Invalid login credentials are rejected	Source inspection	UnauthorizedException on bcrypt.compare failure
Non-conductor cannot accept a consignment	Source inspection	@Roles('CONDUCTOR') on POST /consignments/:id/accept
Pickup/dropoff halts must belong to selected route	Source inspection	BadRequestException in ConsignmentsService.create
Client-supplied fare is ignored	Source inspection	resolveDemoFare() always used, DTO fare field discarded
Handover cannot start outside ACCEPTED	Source inspection	BadRequestException in initiateHandover
Verification cannot succeed outside IN_TRANSIT	Source inspection	BadRequestException in verifyHandover
Only the assigned recipient can verify a PIN	Source inspection	ForbiddenException on recipientId mismatch
Expired PIN is rejected	Source inspection	expiresAt compared to current time before hash check
PIN attempts are capped and counted	Source inspection	MAX_ATTEMPTS = 5; attempts incremented on mismatch
Admin cannot mutate consignments	Source inspection	No @Roles('ADMIN') on any consignment write endpoint
Core consignment lifecycle logic	Automated (repo)	consignments.service.spec.ts – 39 cases, not re-executed

No end-to-end manual run of the mobile app or dashboard against a live database was performed in this documentation session. Performance benchmarking was not conducted as part of the current prototype.

Results & Prototype Demonstration

No screenshots are included in this document. The prototype requires a provisioned PostgreSQL instance, applied migrations and seed data to render any authenticated screen; reliably standing that up plus a mobile (Expo) and web (Next.js) runtime inside this documentation session was not attempted rather than risk fabricated or misleading captures. The screen inventory on Page 5 and the flow/diagram on Page 4 describe the same surfaces a screenshot set would otherwise show.

Implemented outcomes (verified against repository)

- Four-role authentication and authorization (Sender, Conductor, Recipient, Admin) enforced server-side.
- Route → halt → bus network model, populated through both manual admin entry and an OSM ingestion script.
- Full sender booking flow: route browse → halt selection → parcel details → tracking-code issuance.
- Conductor accept → handover flow with server-generated, hashed, expiring PIN.
- Recipient PIN verification with attempt limiting, transitioning the consignment to DELIVERED.
- Admin network management (routes/halts/buses) and single-consignment lookup by tracking ID.

Current limitations (verified)

- No live GPS position or ETA — no location field exists on Bus/Route.
- No exposed event-timeline endpoint — ConsignmentEvent rows are written on every transition but never returned by any API response.
- No PIN regeneration/resend endpoint — a lost or expired PIN has no in-app recovery path.
- No admin-wide consignment list or aggregate dashboard endpoint — admin lookup is single-record, by ID only.
- No recipient search — senders enter a recipient's raw account ID; no name/phone lookup endpoint exists.
- No offline-first storage or sync layer in the mobile client, despite that being the product's stated direction.
- IncentiveLedger exists in the schema with no reading/writing backend module — modeled, not wired up.

Demo flow

Login → Book (route/halt/parcel) → Accept (conductor) → Initiate handover (PIN issued) → Verify (recipient enters PIN) → DELIVERED

Future Scope & References

Future scope (not implemented — proposed direction only)

- Live bus position and ETA prediction, once a location/telemetry source is available.
- A queryable event-timeline API surfacing the already-recorded ConsignmentEvent history.
- Notification infrastructure (SMS/push) for status changes and PIN issuance.
- Recipient lookup by phone/name to replace raw-ID entry during booking.
- PIN regeneration/recovery for lost or expired handover codes.
- Admin-wide consignment search, filtering, and aggregate operational dashboards.
- Offline-first mobile storage and sync (the architecture direction named in the project README).
- Route optimization and production-scale deployment hardening.

Technical references

- NestJS — <https://docs.nestjs.com>
- Prisma ORM — <https://www.prisma.io/docs>
- PostgreSQL — <https://www.postgresql.org/docs>
- Expo / React Native — <https://docs.expo.dev>
- React Navigation — <https://reactnavigation.org/docs/getting-started>
- Next.js — <https://nextjs.org/docs>
- Passport JWT strategy — <https://www.passportjs.org>

Project status

Implemented	Demonstrated workflow	Known limitation
4-role auth, route/halt/bus model, full booking-to-delivery lifecycle, PIN-based handover, admin network CRUD	Login → Book → Accept → Handover → Verify → Delivered	No offline mode, no live tracking, no event-timeline API, no PIN recovery, single-record admin lookup only

This document reflects the state of the Vahak repository at the time of writing. Where the product README or design materials describe capabilities beyond what is implemented (offline-first sync, QR scanning, live event timelines, incentive payouts), those are marked above as future scope, not current functionality.